MINI PROJECT 3
Real-Time Weather API Data Analysis

Fetching, Cleaning & Visualizing Live Data for 10 Indian Cities

By Saumitra Purkayastha ; USN : 23BTRCL113

Report Date	April 21, 2026
Project Type	API Data Analysis & Visualization
Data Source	OpenWeatherMap API (Real-time)
Cities Analyzed	10 major Indian cities
Variables Captured	15+ weather metrics
Visualizations	9 comprehensive charts

1. Executive Summary

This project demonstrates a complete real-time data pipeline that fetches live weather data from the OpenWeatherMap API, stores it in a structured Pandas DataFrame, performs comprehensive data cleaning and feature engineering, and generates 9 meaningful visualizations with statistical analysis. The analysis reveals clear patterns in temperature distribution across Indian cities, strong correlations between atmospheric variables, and practical insights into humidity-comfort relationships. All work was performed using Python with libraries including Requests, Pandas, Matplotlib, and Seaborn, executed entirely in a Jupyter Notebook environment.

2. Project Objectives & Scope

Data Collection: Fetch real-time weather data for 10 major Indian cities using the OpenWeatherMap REST API

Data Storage: Store 15+ weather variables per city in a structured Pandas DataFrame

Data Cleaning: Validate data quality, check for missing values, detect and handle outliers

Feature Engineering: Create derived features (Heat Index, Comfort Level categories) from raw variables

Statistical Analysis: Compute descriptive statistics, correlations, and identify key patterns

Visualization: Generate 9 diverse chart types to communicate insights effectively

3. Data Collection

3.1 Cities Analyzed

The project analyzed real-time weather data from 10 major Indian cities, selected to represent diverse geographic regions and climate zones across the country:

• Bengaluru (South India, Coastal-adjacent)

• Mumbai (West Coast, Coastal)

• Delhi (North India, Inland Plains)

• Chennai (South Coast, Coastal)

• Kolkata (East India, Riverine)

• Hyderabad (South-Central, Inland)

• Pune (West-Central, Elevated)

• Ahmedabad (Northwest, Inland)

• Jaipur (North, Desert Region)

• Lucknow (North-Central, Inland Plains)

3.2 Variables Captured (15+)

Variable	Type	Description
Temperature (°C)	Float	Current air temperature in Celsius
Feels Like (°C)	Float	Perceived temperature accounting for wind & humidity
Min/Max Temp (°C)	Float	Daily temperature range boundaries
Humidity (%)	Integer	Relative humidity percentage
Pressure (hPa)	Integer	Atmospheric pressure at sea level
Wind Speed (m/s)	Float	Wind speed in metres per second
Wind Direction (°)	Integer	Wind direction in degrees (0-359)
Visibility (km)	Float	Horizontal visibility distance
Cloudiness (%)	Integer	Cloud cover percentage
Weather Condition	String	General weather description (Clear, Clouds etc.)
Sunrise/Sunset	Time	Local sunrise and sunset times

4. Methodology

4.1 API Integration

Data was fetched using the OpenWeatherMap REST API (Free Tier). For each city, an HTTP GET request was sent to the *weather/2.5/weather* endpoint with parameters: city name, API key, and units=metric (Celsius). Responses were parsed from JSON format and extracted into Python dictionaries, with error handling to gracefully skip unavailable cities.

4.2 Data Processing Pipeline

Step 1 - Fetch: requests.get() → JSON response for each city

Step 2 - Parse: response.json() → extract main, wind, clouds, sys objects

Step 3 - Collect: Accumulate 10 dictionaries into a list

Step 4 - Frame: pd.DataFrame(list) → tabular structure (10 rows × 20+ columns)

Step 5 - Validate: Check for nulls, outliers using IQR method
Step 6 - Engineer: Create Heat Index and Comfort Level columns
Step 7 - Analyze: Compute statistics, correlations, groupings
Step 8 - Visualize: Generate 9 chart types using Matplotlib/Seaborn

4.3 Feature Engineering

Two new features were created from raw data: **Heat Index:** Combines temperature and humidity using the formula: Heat Index = Temp + 0.33 × (Humidity/100 × 6.105) - 4. This provides a realistic estimate of how hot it feels when accounting for moisture in the air. **Comfort Level:** Categorical variable classifying each city into one of four categories based on temperature and humidity thresholds:

Category	Temperature Range	Humidity Constraint	Description
Cool	< 20°C	N/A	Pleasant, cool conditions
Comfortable	20–28°C	< 60%	Ideal comfort range
Warm	28–32°C	N/A	Warm but manageable
Hot & Humid	> 32°C	> 70%	Uncomfortable, high heat stre

ss

5. Key Findings & Insights

5.1 Temperature Distribution

Analysis of temperature data across 10 cities reveals significant variation:

• **Hottest Cities:** Jaipur (40.2°C) and Ahmedabad (39.5°C) experience extreme heat, typical of northwestern India's desert climate.

• **Coolest Cities:** Bengaluru (27.3°C) and Pune (30.4°C) maintain moderate temperatures due to elevation and geographic position.

• **Temperature Spread:** 13°C difference between hottest and coolest cities, reflecting India's diverse climate zones.

• **Pattern:** Inland northern and western cities show higher temperatures; coastal and elevated southern cities are cooler.

5.2 Humidity-Comfort Relationships

Humidity plays a critical role in perceived comfort:

• **Coastal Cities:** Mumbai (85%), Chennai (80%), Kolkata (78%) consistently exceed 70% humidity despite moderate temperatures, causing significant "feels hotter" effect.

• **Inland Cities:** Jaipur (18%), Ahmedabad (22%), Delhi (25%) have low absolute humidity but high actual temperatures, making them extremely hot.

• **Perception Gap:** Cities with high humidity show larger differences between actual and "feels like" temperatures up to 4–5°C in coastal regions.

• **Comfort Zones:** Only Bengaluru, Pune qualify as "Comfortable"; most others are either "Warm" or "Hot & Humid."

5.3 Wind Impact & Pressure Patterns

• **Windiest Cities:** Chennai (6.0 m/s) and Ahmedabad (6.8 m/s) experience strong winds, partly due to coastal and desert geography.

• **Wind Cooling Effect:** Higher wind speeds correlate with larger perception gaps—wind chill is measurable in data.

• **Pressure Variation:** Range from 998 hPa (Ahmedabad) to 1012 hPa (Bengaluru), mild variation across sea-level regions.

• **Visibility:** Generally good (6–10 km) across all cities; coastal cities show slightly lower visibility (7–8 km) due to salt moisture.

5.4 Statistical Correlations

Correlation analysis reveals variable relationships:

Temperature ↔ Heat Index: r = +0.97 (very strong positive) - Heat Index formula is directly derived from temperature.

Temperature ↔ Humidity: r = –0.42 (moderate negative) — Hotter inland cities tend to be drier.

Temperature ↔ Pressure: r = –0.28 (weak negative) — Higher temperatures slightly associated with lower pressure.
Humidity ↔ Cloudiness: r = +0.55 (moderate positive) More moisture correlates with more cloud cover. These patterns align with meteorological principles and validate the data quality.

6. Visualizations & Data Representation

#	Chart Type	Purpose	Key Insight
1	Bar Chart	Temperature across cities	Shows geographic temperature variation
2	Grouped Bar	Actual vs Feels Like	Reveals perception gaps due to humidity/wind
3	Horizontal Bar	Humidity levels	Identifies high-humidity cities with comfort threshold
4	Scatter Plot	Temp vs Humidity	Visualizes comfort zones and outliers
5	Bar Chart	Wind speeds	Identifies windiest regions
6	Heatmap	Correlation matrix	Shows variable relationships
7	Range Plot	Daily temp range	Displays min/max/actual per city
8	Pie + Bar	Conditions & comfort	Distribution of weather types and comfort levels
9	Line + Bar	Pressure & visibility	Atmospheric and visibility patterns

7. Technical Implementation

7.1 Libraries & Tools

Python 3.10+ — Core language **Requests** — HTTP client for API calls

Pandas — Data manipulation and DataFrame operations

Matplotlib — Base charting library

Seaborn — Statistical visualization enhancements

NumPy — Numerical operations (correlations, quantiles)

Jupyter Notebook — Interactive development environment

OpenWeatherMap API — Real-time weather data source

7.2 Code Structure

The Jupyter Notebook is organized into 9 sequential cells: 1. **Imports:** Load all required libraries and set plot styles 2. **API Config:** Define API key, base URL, and city list 3. **Fetch Function:** Define fetch_weather() with error handling 4. **Data Collection:** Loop through cities and fetch live data 5. **DataFrame Creation:** Convert list of dicts to Pandas DataFrame 6. **Data Cleaning:** Validate nulls, detect outliers, engineer features 7. **Statistical Summary:** Compute describe(), correlations, key metrics 8. **Visualizations:** Generate 9 charts with labels and formatting 9. **Export:** Save DataFrame to CSV for future reference This structure demonstrates professional data science workflow: fetch → store → clean → analyze → visualize.

7.3 Error Handling

Robust error handling ensures the pipeline doesn't crash: • **API Timeouts:** 10-second timeout on requests prevents indefinite hangs • **Authorization Errors:** 401 errors caught and skipped (key not yet active) • **Missing Fields:** Safe .get() method retrieves optional fields with defaults • **Network Failures:** try-except blocks catch RequestException for any network issue • **Data Validation:** IQR method identifies temperature outliers for review This design allows the analysis to proceed even if 1–2 cities fail to respond.

8. Skills Demonstrated

API Integration: Authenticating with external REST APIs, parsing JSON responses, handling HTTP errors

Data Collection: Looping over multiple resources, accumulating data into structured formats

Data Cleaning: Null/outlier detection using IQR method, data validation techniques

Feature Engineering: Deriving new columns from existing data (Heat Index, Comfort Level classification)

Statistical Analysis: Descriptive statistics, correlation analysis, quartile-based distributions
Data Visualization: Bar, grouped bar, horizontal bar, scatter, heatmap, pie, line, range plots
Pandas Mastery: DataFrame construction, column operations, .apply(), .groupby(), .describe()
Matplotlib/Seaborn: Plot customization, colors, legends, annotations, axis labeling
Python Programming: Functions, loops, list comprehensions, exception handling, datetime operations

Jupyter Notebooks: Interactive analysis, markdown documentation, cell organization

Data Storytelling: Selecting meaningful visualizations, interpreting patterns, communicating insights

9. Conclusions & Recommendations

This project successfully demonstrates a complete real-time data analysis pipeline, from API integration through visualization. Key achievements include:

- Real-time data collection from live weather API without pre-downloaded datasets
- Proper data validation and outlier detection
- Feature engineering that adds practical value (Heat Index, Comfort categories)
- 9 diverse visualizations revealing meaningful patterns
- Statistical rigor with correlation analysis and descriptive statistics

The analysis reveals clear geographic patterns in Indian weather: northwestern cities experience extreme heat with low humidity; coastal cities have moderate temperatures but high humidity; southern elevated cities maintain the most comfortable conditions. These insights have practical applications for urban planning, HVAC design, agriculture, and public health preparedness.

Future Enhancements:

- **Time Series Analysis:** Collect data at hourly intervals over weeks to analyze daily/seasonal trends
- **Forecasting:** Build predictive models (ARIMA, Prophet) for temperature/humidity forecasts
- **Weather Events:** Alert system for extreme conditions (heat waves, high humidity thresholds)
- **Comparison with Historical:** Fetch 5-year averages from OpenWeatherMap and compare current conditions
- **Interactive Dashboard:** Build live Plotly/Streamlit dashboard for real-time monitoring
- **Regional Clustering:** K-means clustering to identify cities with similar weather profiles
- **API Scaling:** Extend to 100+ global cities for continental weather comparison